SavorSync: A Culturally-Aware Meal Planning Agent Using Search and Optimization

Xinjie Shen
CS5100 Final Project
April 2026

1. Introduction

Meal planning is something people do every week, but it is more complicated than simply choosing a few recipes. In practice, a strong meal plan needs to balance several competing factors at the same time, including nutrition, preparation time, ingredient accessibility, variety, and personal preferences. In addition, many people are interested in trying dishes from different cultural cuisines, which adds another layer of difficulty. While many recipe tools focus on recommending individual dishes, they usually do not address the larger problem of how an entire weekly meal plan should be constructed and optimized as a whole.

This project is inspired by SavorSync, an earlier HCI project I worked on that focused on recipe exploration, cultural storytelling, and ingredient substitution. That project helped users learn about and cook dishes from a range of cultural cuisines, especially when traditional ingredients were difficult to find. However, SavorSync mainly worked at the level of individual recipes. In

this project, I wanted to take that idea further and explore a broader AI question: how can an intelligent system build and optimize a weekly meal plan that balances cultural authenticity with practical real-world constraints?

To answer this question, I model weekly meal planning as a combinatorial search and optimization problem. Instead of treating recipes independently, the system represents an entire week of meals as a single search state. A utility-based agent then evaluates candidate plans using a weighted utility function that combines multiple objectives, including nutrition, preparation time, ingredient accessibility, cultural authenticity, cuisine diversity, and repetition. Search algorithms are used to move through the space of possible meal plans and improve them over time.

This framing is supported by prior academic work. Türkmenoğlu et al. (2021) approached meal planning as a many-objective optimization problem and used evolutionary algorithms to generate balanced diet solutions under multiple constraints. Similarly, Ramos-Pérez et al. (2021) showed that balanced menus can be generated using multi-objective evolutionary algorithms, while Salloum and Tekli (2022) treated meal planning as a structured allocation problem under nutritional and preference constraints. These studies suggest that meal planning is naturally suited to optimization-based AI methods because it requires balancing several goals at once rather than optimizing a single factor in isolation. Chaudhary and Krishna (2019) further showed that contextual factors such as geography and culture can also be incorporated directly into diet

optimization models, which supports the decision in this project to include cultural authenticity and ingredient accessibility as part of the evaluation function.

My project builds on this general idea, but differs in two important ways. First, it explicitly includes cultural authenticity and cuisine diversity as optimization objectives within a single weighted utility function. Second, instead of focusing only on evolutionary optimization, it compares three different search paradigms on the same planning problem: hill climbing as a greedy local search baseline, simulated annealing as a stochastic local search method, and a genetic algorithm as an evolutionary approach. This also connects to the broader theory of local search in combinatorial optimization, where iterative improvements over neighboring states can produce strong approximate solutions (Orlin et al., 2004).

The main contribution of this project is therefore a comparative study of how different search strategies behave on a culturally-aware meal-planning task. By comparing greedy local improvement, stochastic exploration, and evolutionary search under the same utility framework, this project aims to better understand how AI search methods can support realistic and personalized meal planning.

2. Methods

This project models weekly meal planning as a search and optimization problem. Instead of treating each recipe recommendation independently, the system represents a full weekly meal

plan as a single state and then applies search algorithms to improve it. The main idea is that a useful meal plan should not only contain individually good recipes, but should also work well as a complete set. To make this possible, I built a planner that starts from an initial random plan, scores it using a weighted utility function, and then improves it through search.

A state in this system is a MealPlan object containing 10 recipes chosen from the recipe dataset. Each recipe includes attributes such as cuisine, preparation time, ingredient accessibility, cultural authenticity, protein, vegetables, and calories. I used a fixed-size meal plan because it makes the search space easier to define and compare across algorithms. In addition to weighted preferences, I also used hard constraints to remove obviously weak plans from consideration. For example, if a meal plan does not meet minimum nutrition requirements or does not contain enough diversity, it is assigned a score of negative infinity. In practice, this means the plan is treated as invalid and rejected by the search process.

Valid meal plans are scored using a weighted utility function. The scoring function combines several factors that reflect the overall quality of a weekly meal plan: total protein, total vegetables, ingredient accessibility, cultural authenticity, cuisine diversity, preparation time, and repetition. Higher protein and vegetable totals generally improve the score, while long preparation times and repeated meals lower it. This design allows the system to balance practical concerns with cultural and nutritional goals rather than optimizing only one factor. To make the planner more personalized, I implemented three different preference modes by adjusting the weights in the utility function. In health-focused mode, the planner rewards nutrition more

strongly. In convenience-focused mode, preparation time and accessibility matter more. In cultural-exploration mode, cuisine diversity and authenticity are emphasized more heavily. This lets the same search framework produce different kinds of meal plans depending on the user's priorities.

To move through the search space, the system generates neighboring meal plans by making small changes to the current state. These neighbor operations define the action space of the problem. Some actions are general, such as replacing one meal with another or swapping two meals. Others are more targeted and try to improve a specific weakness in the current plan, such as replacing the longest-prep meal with a faster option, replacing a low-vegetable meal with a higher-vegetable alternative, or substituting a recipe with one from a culturally related cuisine. I also refined the neighbor-generation logic so that it avoids unnecessary duplicates whenever possible and prefers unused recipes when making replacements. These design choices make the search more meaningful because the planner is not just making random changes, but changes that are more closely tied to the goals of the problem.

I implemented three different search algorithms and compared them on the same meal-planning task. The first was hill climbing, which serves as a greedy local-search baseline. Starting from a random initial plan, hill climbing repeatedly generates a neighbor and accepts it only if it improves the utility score. This makes the algorithm simple and efficient, but it also means it can easily get stuck in local optima. The second method was simulated annealing, which extends local search by occasionally accepting a worse state with a probability controlled by temperature.

Early in the search, this allows the algorithm to explore more broadly and escape poor local optima. As the temperature decreases, the behavior becomes more conservative and eventually resembles hill climbing. The third method was a genetic algorithm, which provides an evolutionary baseline and also connects more directly to the meal-planning optimization literature reviewed earlier. In this representation, a full weekly meal plan acts as a chromosome and individual meals act as genes. I used tournament selection to choose parent plans, crossover to combine the first half of one plan with the second half of another, and mutation through the same neighbor-generation logic used by the other algorithms. I also added elitism, meaning that the best solution from each generation is copied directly into the next generation so that strong solutions are not lost during crossover or mutation. This made the genetic algorithm more stable and easier to analyze. In the final implementation, hill climbing was run for 500 iterations. Simulated annealing was run for 1000 iterations with an initial temperature of 10.0 and a cooling rate of 0.995. The genetic algorithm used a population size of 30, 50 generations, tournament selection with k = 3, and a mutation rate of 0.2.

To evaluate the system, I ran repeated experiments for all three algorithms under all three preference modes. Because the algorithms include randomness in initialization and search behavior, I ran each condition multiple times and compared average scores rather than relying on a single run. The main evaluation measures were final utility score, average score across runs, and number of valid runs. In addition to these numerical results, I also tracked score history over time for each algorithm and generated convergence plots in Python using matplotlib (Hunter, 2007). These plots made it possible to compare how quickly hill climbing improved, how

simulated annealing explored more variably, and how the genetic algorithm evolved over generations. Together, these experiments were designed to evaluate not only which method produced the strongest meal plans on average, but also how their search behavior differed across the same optimization problem.

3. Results

To evaluate the performance of the three search algorithms, I ran repeated experiments under each preference mode. Because all three methods involve randomness, including random initialization and stochastic search decisions, each algorithm-mode combination was executed five times and the resulting utility scores were averaged. This makes the evaluation more reliable by reducing the influence of any single unusually strong or weak run.

Across all experiments, all three algorithms consistently improved the quality of meal plans compared to the initial random plans. This suggests that the search formulation and utility function are effective for guiding optimization toward stronger weekly meal plans. To illustrate the system's behavior more concretely, a sample output from a single run in health-focused mode is shown below.

Example Output (Health-Focused Mode):

```
INITIAL RANDOM PLAN:
BEAN BURRITO BOWL, VEGETABLE CURRY, CHICKEN WRAP, ETHIOPIAN LENTIL PLATE, ...
```

```
FINAL SCORE: 269.91

HILL CLIMBING RESULT:
MEDITERRANEAN SALMON PLATE, CHICKEN TIKKA BOWL, GREEK CHICKEN PLATE, ...
FINAL SCORE: 367.56

SIMULATED ANNEALING RESULT:
PERUVIAN CHICKEN BOWL, MEDITERRANEAN SALMON PLATE, CHICKEN STIR FRY, ...
FINAL SCORE: 366.76

GENETIC ALGORITHM RESULT:
GRILLED CHICKEN SALAD, THAI BASIL CHICKEN, MEDITERRANEAN SALMON PLATE, ...
FINAL SCORE: 364.99

BEST ALGORITHM IN THIS RUN: HILL CLIMBING
```

As this example shows, all three algorithms substantially improve the starting random plan, with hill climbing and simulated annealing achieving the highest scores in this particular run.

The repeated experiments showed a more complete picture of algorithm performance. In health-focused mode, simulated annealing achieved the strongest average score at 368.81, compared to 366.36 for hill climbing and 362.14 for the genetic algorithm. In convenience-focused mode, hill climbing achieved the highest average score at 196.88, slightly above simulated annealing at 196.64 and the genetic algorithm at 195.73. In cultural-exploration mode, hill climbing again performed best on average with 226.48, followed closely by simulated annealing at 226.24, while the genetic algorithm averaged 225.34.

These results show that hill climbing and simulated annealing are both highly effective on this problem, with the stronger method depending somewhat on the preference mode. Simulated

annealing performed best in the health-focused setting, which is also the most multi-objective of the three modes, since it requires balancing nutrition, diversity, and practicality at the same time. Hill climbing performed slightly better in the convenience-focused and cultural-exploration settings, suggesting that a greedy strategy can still work well when the search space is either simpler or more strongly shaped by one dominant direction of improvement. The genetic algorithm consistently generated valid and fairly strong meal plans, but under the current implementation it remained slightly below the two local-search methods in all three modes.

To better understand how the algorithms behaved during optimization, I also generated convergence plots that track utility scores over time for each method. Figures 1–3 show the convergence behavior of hill climbing, simulated annealing, and the genetic algorithm in the health-focused, convenience-focused, and cultural-exploration modes.

Figure 1. Convergence plot for health-focused mode

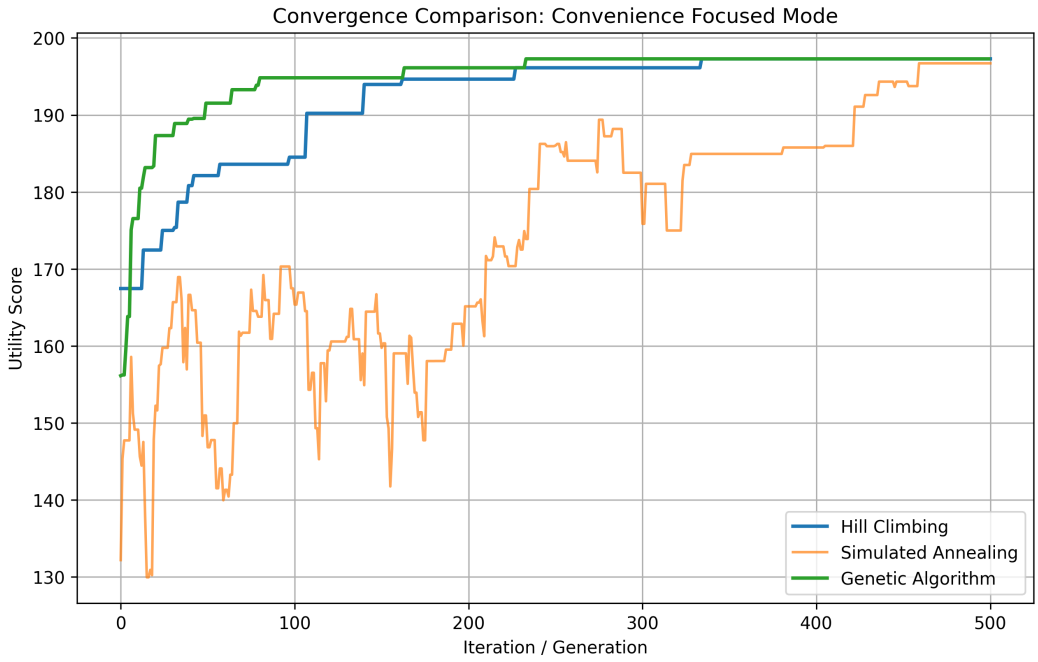


Figure 2. Convergence plot for convenience-focused mode

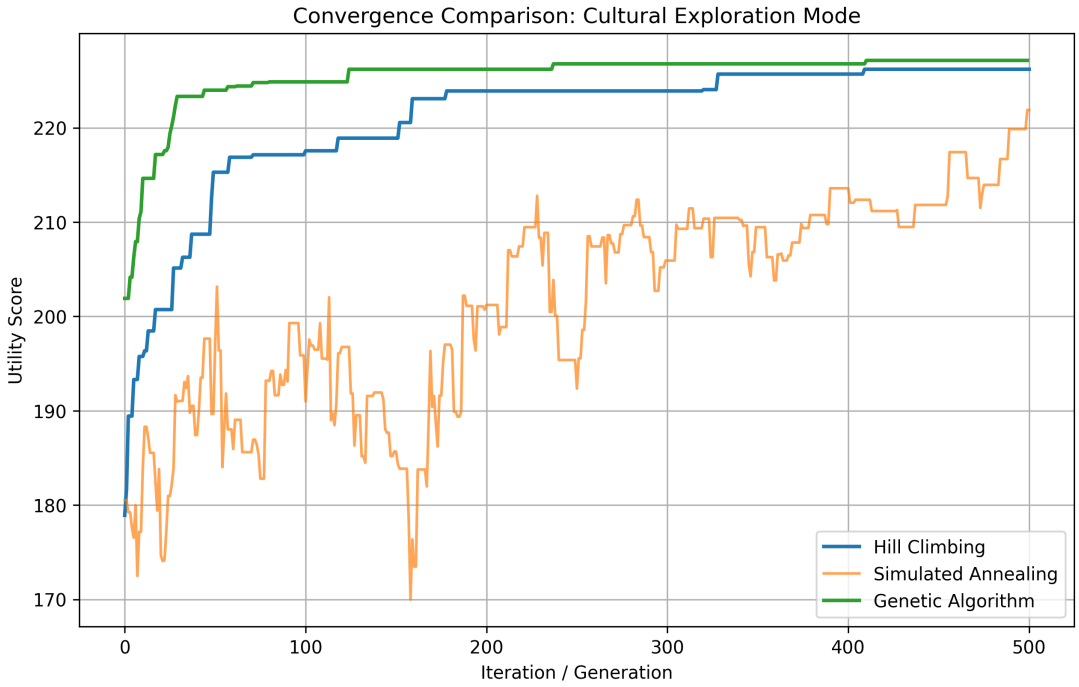
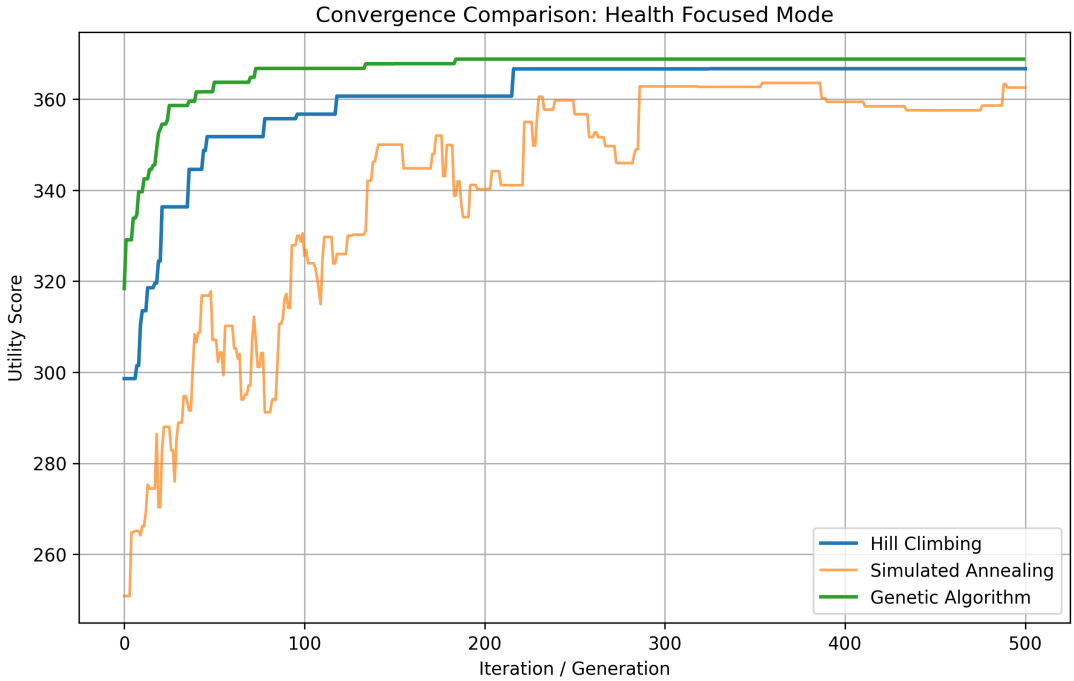


Figure 3. Convergence plot for cultural-exploration mode



Convergence Analysis

The convergence plots provide additional insight into how the three algorithms explore and improve meal plans over time.

Hill climbing shows very rapid improvement in the early iterations, visible as a steep initial increase in score. However, its curve quickly flattens, indicating that the algorithm often becomes trapped in a local optimum and is unable to continue improving. This pattern appears consistently across all three preference modes and matches the expected behavior of greedy local search.

Simulated annealing behaves differently. Its curve is much more variable in the early stages, sometimes dropping before rising again. This happens because the algorithm occasionally accepts worse intermediate states, which allows it to explore a larger portion of the search space instead of committing too early to a locally good solution. Over time, the curve stabilizes and often reaches slightly better final scores than hill climbing, especially in the health-focused setting, suggesting that this added exploration helps simulated annealing escape local optima more effectively.

The genetic algorithm shows a third pattern. It tends to improve quickly in the early generations and then stabilize at a relatively high score. This step-like behavior reflects its population-based structure and the use of elitism, which preserves the best solution from one generation to the

next. At the same time, the curve often plateaus earlier than simulated annealing, suggesting that while elitism improves stability, it may also reduce diversity and limit later improvement.

Taken together, these results highlight a clear exploration-exploitation trade-off across the three methods. Hill climbing is efficient but easily trapped in local optima. Simulated annealing balances exploration and exploitation more effectively, which helps explain why it performed best in the most complex setting. The genetic algorithm provides a meaningful evolutionary baseline and stable convergence behavior, but under the current implementation it does not yet consistently outperform the local-search methods.

4. Discussion and Conclusions

Overall, this project shows that weekly meal planning can be modeled effectively as a search and optimization problem. Representing a full weekly meal plan as a single search state worked well because it allowed the system to evaluate meals not only individually, but also as part of a larger plan. This made it possible to optimize for multiple goals at once, including nutrition, preparation time, accessibility, cultural authenticity, cuisine diversity, and repetition. One of the strongest outcomes of the project is that the same utility-based framework was able to produce meaningfully different meal plans simply by changing the preference weights. In this sense, the project successfully demonstrates that a single AI planner can support different user goals, such as health, convenience, or cultural exploration, without changing the overall structure of the system.

The comparison across algorithms also produced useful results. Hill climbing worked as a strong greedy baseline and often found high-scoring plans quickly, but its convergence plots showed that it tended to plateau early. This is consistent with the main weakness of greedy local search, since it can easily become trapped in local optima. Simulated annealing was especially effective in the health-focused mode, where it achieved the highest average score and benefited from its ability to accept worse intermediate states and explore the search space more broadly. However, hill climbing slightly outperformed simulated annealing in the convenience-focused and cultural-exploration modes, while simulated annealing remained very close. These results suggest that both local-search methods are highly effective, with simulated annealing offering an advantage in more complex optimization settings and hill climbing remaining competitive in settings where a simpler greedy strategy is sufficient. The genetic algorithm added an important evolutionary baseline and made the project stronger as a comparative study, but under the current implementation it generally underperformed compared to the best local-search results. Even with elitism, the genetic algorithm tended to converge conservatively and stabilize earlier than simulated annealing.

There are several limitations that help explain these results. First, the genetic algorithm likely did not receive enough search budget to fully compete with the local-search methods. A relatively small population and limited number of generations may not provide enough opportunities for crossover and mutation to build stronger solutions than fast iterative local search. Second, the crossover operator is still fairly simple, since it combines plans by splitting them in half rather

than preserving especially strong meal combinations in a more intelligent way. Third, the cultural substitution logic is still heuristic rather than knowledge-based. Recipes are grouped by related cuisines, which works as a simple approximation, but it does not capture ingredient-level cultural reasoning. Finally, the utility function relies on fixed hand-designed weights rather than learned or adaptive preferences, which means the planner is still limited by the assumptions built into its scoring design.

Even with these limitations, the project demonstrates a meaningful and complete AI framework. It combines state representation, utility-based evaluation, successor-function design, local search, evolutionary search, repeated experiments, and convergence analysis in a single system. It also shows that algorithmic differences matter in practice: hill climbing is efficient but greedy, simulated annealing is more exploratory and flexible, and the genetic algorithm is stable but requires more tuning to reach its full potential. Together, these results make the project more than just a meal generator; it becomes a comparative study of how different search strategies behave on a realistic multi-objective planning problem.

There are several clear directions for future work. One improvement would be to scale the recipe dataset significantly so the planner can operate in a much richer search space. Another would be to design more advanced crossover and mutation operators for the genetic algorithm so that strong meal combinations are preserved more effectively. The planner could also be extended with more realistic scheduling constraints, such as modeling breakfast, lunch, and dinner separately or accounting for meal order across days. Finally, future versions could move beyond

fixed preference modes by allowing dynamic user input or learned preference weights. These improvements would make the system more realistic, more personalized, and more competitive as a larger optimization framework.

5. Works Cited

Chaudhary, A., & Krishna, V. (2019). Country-specific sustainable diets using optimization algorithm. *Environmental Science & Technology, 53*(13), 7694–7703. https://doi.org/10.1021/acs.est.8b06923

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Orlin, J. B., Punnen, A. P., & Schulz, A. S. (2004). Approximate local search in combinatorial optimization. *SIAM Journal on Computing, 33*(5), 1201–1214. https://doi.org/10.1137/S0097539703431007

Ramos-Pérez, J.-M., Miranda, G., Segredo, E., León, C., & Rodríguez-León, C. (2021). Application of multi-objective evolutionary algorithms for planning healthy and balanced school lunches. *Mathematics, 9*(1), 80. https://doi.org/10.3390/math9010080

Salloum, G., & Tekli, J. (2022). Automated and personalized meal plan generation and relevance scoring using a multi-factor adaptation of the transportation problem. *Soft Computing, 26*, 2561–2585. https://doi.org/10.1007/s00500-021-06400-1

Türkmenoğlu, C., Uyar, A. Ş. E., & Kiraz, B. (2021). Recommending healthy meal plans by optimising nature-inspired many-objective diet problem. *Health Informatics Journal, 27*(4). https://doi.org/10.1177/1460458220976719